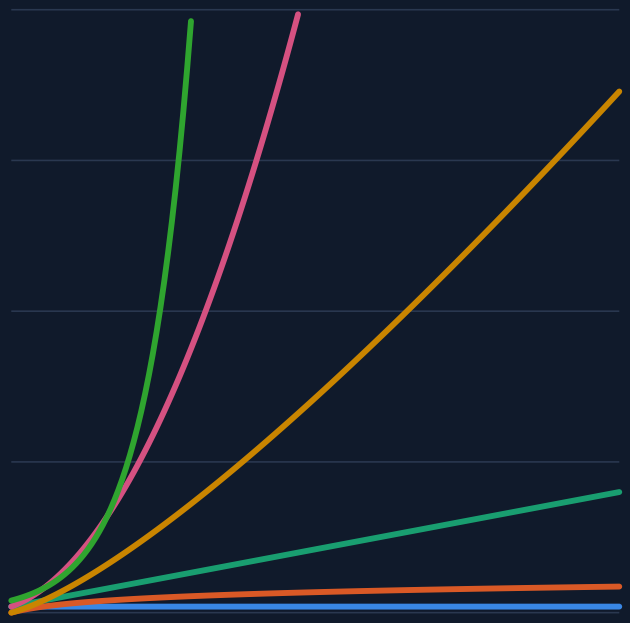
АЛГОРИТМЫН ҮНДЭС · PYTHON

Big O

зөн совин

Цаг ба санах ойн төвөгшил,
өсөлтийн харьцуулалт



Түвшин: Ойлголт | **Жишээ:** Python 3 | **Урьдчилсан мэдлэг:** for давталт, функц

Энэ хичээлийг уншсаны дараа та:

- Big O **юуг** хэмждэг, **юуг** хэмждэггүйг энгийн үгээр тайлбарлаж чадна.
- Python код хараад түүний **цагийн** ба **санах ойн** төвөгшлийг таамаглана.
- O(1), O(log n), O(n), O(n log n), O(n²), O(2ⁿ) -ийг өсөлтийн хурдаар нь эрэмбэлнэ.
- Хурд ба санах ойн хоорондох **солилцоог** (trade-off) ойлгоно.

Агуулга

01 Big O гэж юу вэ?
02 Алхам тоолох ба хялбарчлах дүрэм
03 Түгээмэл 6 ангилал
04 Өсөлтийн харьцуулалт
05 Python дээр өөрсдөө хэмжиж үзье
06 Санах ойн төвөгшил
07 Цаг ба санах ойн солилцоо
08 Python үйлдлүүдийн хуудас
09 Түгээмэл андуурал
10 Дасгал
11 Хураангуй

> **Нэг өгүүлбэрээр**
> Big O нь «оролт томрох тусам програмын ажил хэр **хурдан өсөх** вэ» гэсэн асуултад хариулдаг.

01 Big O гэж юу вэ?

Нэг асуудлыг шийддэг хоёр програм байлаа гэж бодъё. Аль нь илүү вэ? Секундээр хэмжиж харьцуулах нь хамгийн энгийн санаа мэт боловч найдвартай биш: хариу нь компьютер, програмчлалын хэл, тэр үед ажиллаж буй бусад програм гээд олон зүйлээс хамаарна. Big O үүний оронд **өөр асуулт** тавьдаг:

> **Big O-гийн асуулт**
>
> **Оролт томрох тусам ажлын хэмжээ хэрхэн өсөх вэ?**

Жишээ: утасны дэвтрээс нэр хайх

1 000 хуудастай утасны дэвтрээс «Тэмүүлэн» гэдэг нэрийг хайж байна гэж төсөөл. Нэрс цагаан толгойн дарааллаар байгаа.

	Арга А: эхнээс нь нэг бүрчлэн	Арга Б: дундаас нь нээх
Яаж?	Хуудас бүрийг дарааллаар нь харна.	Дунд хуудсыг нээж, нэр өмнө нь уу, хойно нь уу гэдгийг хараад хагасыг нь хаяна. Үлдсэн хагаст дахин давтана.
1 000 хуудас	≈ 1 000 алхам	≈ 10 алхам
2 000 хуудас	≈ 2 000 алхам	≈ 11 алхам
1 000 000 хуудас	≈ 1 000 000 алхам	≈ 20 алхам
Big O	■ **O(n)**	■ **O(log n)**

Дэвтэр 2 дахин зузаарахад А аргын ажил **2 дахин** нэмэгдсэн бол Б аргынх ердөө **1 алхмаар** нэмэгдлээ. Big O яг энэ ялгааг барьж авдаг. Цаашид **n** гэдэг нь **оролтын хэмжээ**: жагсаалтын урт, тэмдэгт мөрийн урт, хуудасны тоо гэх мэт.

> **Гол санаа**
>
> Big O бол **хурд биш, өсөлтийн хэлбэр**. Энэ нь «n томрох үед юу болох вэ» гэдгийг л хэлнэ. Тиймээс бид секунд биш, **алхам** тоолно.

Цаг ба санах ой

Big O-г хоёр зүйлд хэрэглэдэг:

- **Цагийн төвөгшил** (time complexity): n-ээс хамаарч хэдэн алхам хийх вэ?
- **Санах ойн төвөгшил** (space complexity): n-ээс хамаарч хэр их *нэмэлт* санах ой эзлэх вэ?

Хоёулаа ижил дүрмээр тооцогддог. Эхлээд цагийг үзээд, 6-р бүлэгт санах ой руу шилжинэ.

02 Алхам тоолох ба хялбарчлах дүрэм

Big O-г олохын тулд код хэдэн **энгийн алхам** (нэмэх, харьцуулах, утга оноох, индексээр хандах…) хийхийг n-ээр илэрхийлнэ.

```
def niilber(items):
    total = 0              # 1 удаа
    for x in items:        # n удаа эргэнэ
        total += x         # n удаа
    return total           # 1 удаа
```

Нийт ≈ **2n + 2** алхам. Гэвч бид үүнийг зүгээр л **O(n)** гэж бичнэ. Яагаад?

- **«+2» хаягдана:** n = 1 000 000 үед 2 000 002 ба 2 000 000 бараг ялгаагүй. Жижиг нэмэгдэхүүн том n дээр ач холбогдлоо алддаг.
- **«×2» хаягдана:** Энэ коэффициент компьютер, хэлнээс хамаарч өөрчлөгддөг. Харин «n-тэй хамт шулуунаар өсдөг» гэдэг **хэлбэр** нь хаана ч ижил.

5 хялбарчлах дүрэм

#	Дүрэм	Жишээ
1	Тогтмол коэффициентийг хая	O(3n) → O(n); O(500) → O(1)
2	Хамгийн хурдан өсөх гишүүнийг л үлдээ	O(n² + n + 100) → O(n²)
3	Дараалсан хэсгүүдийг **нэм**	давталтын араас давталт: O(n + n) = O(n)
4	Үүрлэсэн (дотор нь) давталтыг **үржүүл**	давталт дотор давталт: O(n · n) = O(n²)
5	Өөр оролтод өөр үсэг	a, b хоёр жагсаалт: O(a + b) эсвэл O(a · b)

3 ба 4-р дүрмийг нэг жишээн дээр:

```
def jishee(items):
    for x in items:            # (1) n алхам
        print(x)

    for x in items:            # (2) гадна давталт: n удаа
        for y in items:        #     дотор давталт: n удаа  → n · n
            print(x, y)

# Нийт: n + n²  →  хамгийн хурдан өсөх гишүүн  →  O(n²)
```

> **Анхаар: 5-р дүрэм**
> Хоёр **өөр** жагсаалт дээр давталт хийвэл O(n²) гэж бичих нь буруу. `for x in a: for y in b:` нь **O(a · b)**. Хэрэв a = 10, b = 1 000 000 бол энэ нь 10 сая, харин O(n²) гэвэл утгагүй их тоо гарна.

03 Түгээмэл 6 ангилал

Ихэнх кодын төвөгшил дараах 6 ангиллын аль нэгд багтдаг. **Сайнаас муу руу** эрэмбэлэв. Ангилал бүрийн өнгө баримт бичиг даяар ижил байна.

O(1) · Тогтмол

Утга: Оролт хэдэн ч том байсан алхмын тоо **өөрчлөгдөхгүй**.

Зүйрлэл: Номын тавиурын дугаар, байрлалыг нь мэддэг номоо шууд очиж авах.

```
def ehnii_element(items):
    return items[0]          # 10 элемент ч, 10 сая ч — 1 алхам

nas = {"Бат": 20, "Сараа": 19}
print(nas["Сараа"])          # dict-ээс түлхүүрээр авах: O(1)
```

n 2 дахин томорвол → ажил **өөрчлөгдөхгүй**.

O(log n) · Логарифм

Утга: Алхам бүрт асуудал **хоёр дахин багасна**. $\log_2$ n гэдэг нь «n-ийг хэдэн удаа 2-т хуваахад 1 болох вэ» гэсэн тоо.

Зүйрлэл: Утасны дэвтрийг дундаас нь нээх; «1-ээс 100-ын хооронд бодсон тоог таа» тоглоом.

```
def binary_search(items, target):     # items эрэмбэлэгдсэн байх ёстой!
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        elif items[mid] < target:
            lo = mid + 1              # зүүн хагасыг хаяна
        else:
            hi = mid - 1              # баруун хагасыг хаяна
    return -1
```

n 2 дахин томорвол → ердөө **+1 алхам**.

$\log_2$ 1 000 000 ≈ 20. Сая элементтэй эрэмбэлэгдсэн жагсаалтаас 20 орчим алхамд олно!



Зураг 1. 16 элементтэй жагсаалтаас 67-г хайх нь. Алхам бүрт хайх хүрээ (цэнхэр) хагасаар багасаж, 4 алхамд олдлоо. Энгийн хайлт бол 11 алхам хийх байсан.

O(n) · Шугаман

Утга: Элемент бүрийг **нэг удаа** (эсвэл тогтмол хэдэн удаа) харна.

Зүйрлэл: Ангийн хүүхэд бүрт нэг нэг чихэр тараах.

```
def hamgiin_ih(items):
    best = items[0]
    for x in items:          # n удаа
        if x > best:
            best = x
    return best
```

n 2 дахин томорвол → ажил **2 дахин**.

O(n log n) · «n лог n»

Утга: Хамгийн сайн ерөнхий **эрэмбэлэх** алгоритмуудын төвөгшил. n элемент бүрийг log n «давхарга»-аар дамжуулна гэж төсөөл.

Зүйрлэл: Хөзрийн багцыг хоёр хувааж, тус бүрийг эрэмбэлээд, буцааж нийлүүлэх (merge sort).

```
onoo = [72, 95, 61, 88, 79]
erembelsen = sorted(onoo)    # Python-ий Timsort: O(n log n)
onoo.sort()                  # байрандаа эрэмбэлэх: мөн O(n log n)
```

n 2 дахин томорвол → ажил **2-оос арай илүү** дахин (≈ 2.2).

$O(n^2)$ · Квадрат

Утга: Элемент бүрийг **бусад бүх** элементтэй харьцуулна. Ихэвчлэн давталт дотор давталт.

Зүйрлэл: Ангийн хүүхэд бүр бусад бүх хүүхэдтэй гар барих.

```
def davhardsan_bii_yu(items):
    n = len(items)
    for i in range(n):                # n удаа
        for j in range(i + 1, n):     #   дотор нь ≈ n/2 удаа
            if items[i] == items[j]:
                return True
    return False
```

n 2 дахин томорвол → ажил **4 дахин**!

n(n−1)/2 харьцуулалт → $n^2/2$ → коэффициентийг хаяхад $O(n^2)$.

$O(2^n)$ · Экспоненциал

Утга: n-ийг **1-ээр** нэмэхэд л ажил **2 дахин** өснө. Бүх боломжит хослолыг шалгахад ихэвчлэн гардаг.

Зүйрлэл: n ширхэг гэрлийн унтраалгын бүх асаах/унтраах хослолыг туршиж үзэх: 2^n хувилбар.

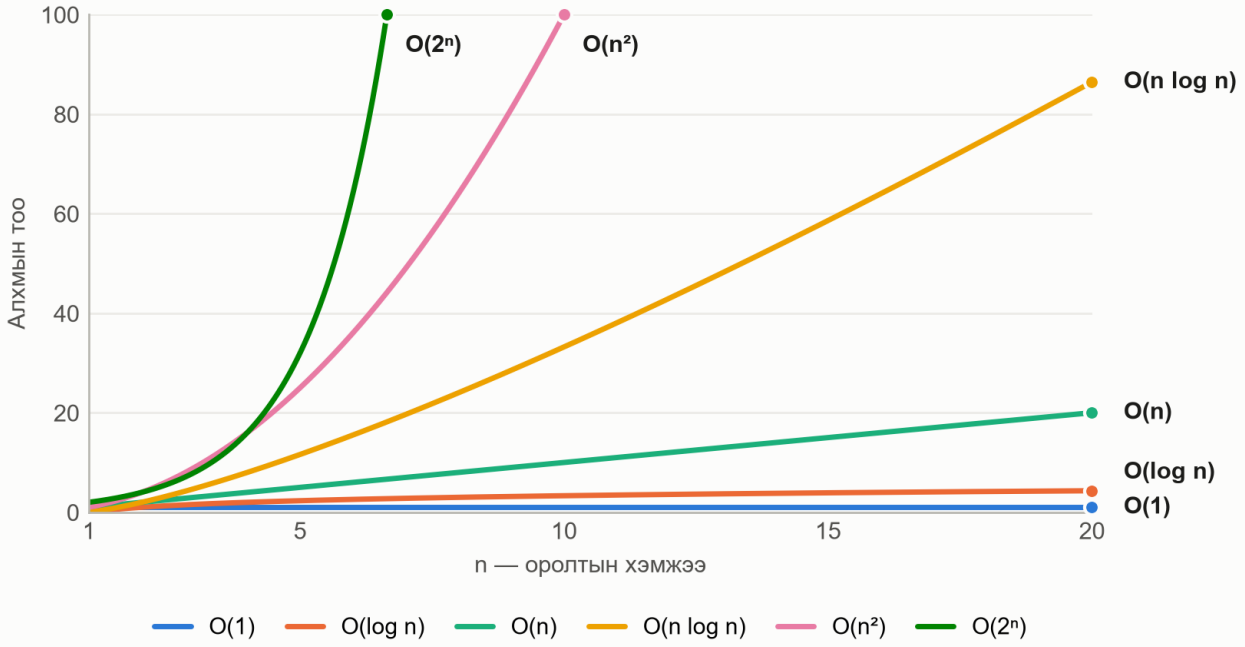
```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)   # дуудлага бүр 2 болж салаална
```

n 2 дахин томорвол → ажил **квадрат зэрэгт** өснө: $2^n \rightarrow 2^{2n} = (2^n)^2$.

fib(40)-ийг ингэж бодоход 331 160 281 удаа функц дуудагдана! 7-р бүлэгт үүнийг хэрхэн хурдасгахыг үзнэ.

04 Өсөлтийн харьцуулалт

Зургийн хэлбэрээр харвал ялгаа нь шууд мэдрэгдэнэ. Доорх график n = 1…20 гэсэн **маш жижиг** оролт дээр ч гэсэн 2^n ба n^2 хэр хурдан «тааз» руу хөөрч байгааг харуулна.



Зураг 2. Алхмын тоо, n = 1…20. Тэнхлэгийг 100-аар таслав: $O(2^n)$ нь n ≈ 6.6, $O(n^2)$ нь n = 10 үед л 100-г давна. Доод талын O(1), O(log n) бараг хэвтээ хэвээр.

Тоогоор харьцуулбал

Ангилал	Нэр	n = 10	n = 100	n = 1 000	n 2 дахин → ажил
O(1)	Тогтмол	1	1	1	өөрчлөгдөхгүй
O(log n)	Логарифм	3	7	10	+1 алхам
O(n)	Шугаман	10	100	1 000	×2
O(n log n)	n лог n	33	664	9 966	×2-оос арай их
O(n^2)	Квадрат	100	10 000	1 000 000	**×4**
O(2^n)	Экспоненциал	1 024	≈ 1.3×10^{30}	≈ 1.1×10^{301}	**квадрат зэрэгт**

Энэ нь хугацааны хувьд юу гэсэн үг вэ?

Компьютер секундэд **100 сая** (10^8) энгийн алхам хийдэг гэж үзээд **n = 1 000 000** элемент боловсруулахад хэр удах вэ гэдгийг тооцъё. (Python үүнээс хэд дахин удаан байж болох ч харьцаа нь ижил.)

Ангилал	Алхмын тоо	Хугацаа
■ **O(1)**	1	агшин зуур
■ **O(log n)**	≈ 20	0.0000002 сек
■ **O(n)**	1 000 000	0.01 сек
■ **O(n log n)**	≈ 20 000 000	0.2 сек
■ **O(n²)**	10^{12}	**≈ 2.8 цаг**
■ **O(2ⁿ)**	≈ $10^{301\,030}$	**Орчлон ертөнцийн наснаас ч хэмжээлшгүй урт**

> **Зөн совин**
> **Сайн алгоритм хурдан компьютерээс илүү чухал.** O(n²) кодыг 100 дахин хурдан компьютер дээр ажиллуулсан ч n хангалттай том болоход O(n log n) код ялна.

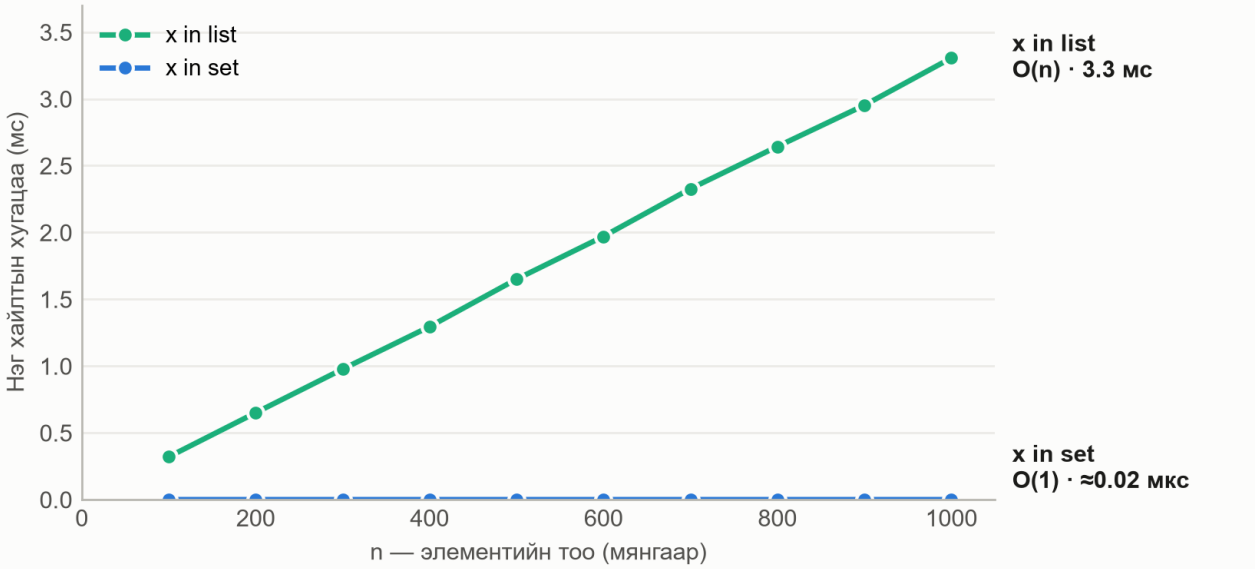
05 Python дээр өөрсдөө хэмжиж үзье

Онол бодит байдалтай таарч байна уу? `x in list` ба `x in set` хоёрыг харьцуулъя. Жагсаалтаас (list) хайхад Python элемент бүрийг дараалаар нь шалгадаг — **O(n)**. Харин олонлог (set) нь *hash хүснэгт* ашиглан тухайн утгын байрлалыг шууд тооцоолж очдог — дунджаар **O(1)**.

```
import timeit

for n in [100_000, 500_000, 1_000_000]:
    a = list(range(n))       # жагсаалт
    s = set(a)               # олонлог (hash хүснэгт)
    target = -1              # байхгүй утга → хамгийн муу тохиолдол

    t_list = timeit.timeit(lambda: target in a, number=20) / 20
    t_set = timeit.timeit(lambda: target in s, number=20) / 20
    print(f"n={n:>9,}  list: {t_list*1000:8.3f} мс   set: {t_set*1000:.5f} мс")
```



Зураг 3. Энэ PDF-ийг үүсгэх үед бодитоор хэмжсэн үр дүн (n = 100 000 … 1 000 000, 5 давталтын хамгийн бага утга).

Манай хэмжилтээр n = 1 000 000 үед list-ээс хайхад **3.31 мс**, set-ээс хайхад **0.023 микросекунд** зарцуулсан — ойролцоогоор **140 000 дахин** ялгаа. Таны компьютер дээрх тоо өөр гарах ч **хэлбэр** нь ижил: list шулуун шугамаар өсөж, set хэвтээ хэвээр байна.

> **Практик зөвлөгөө**
> Олон удаа «байгаа юу?» гэж шалгах бол өгөгдлөө **set** (эсвэл dict) болго. Энэ ганц өөрчлөлт л ихэнхдээ програмыг олон зуу дахин хурдасгадаг.

06 Санах ойн төвөгшил

Санах ойн төвөгшил гэдэг нь алгоритм ажиллахдаа оролтоос **гадна нэмэлтээр** хэр их санах ой эзлэх вэ гэдэг. Дүрэм нь цагийнхтай яг адилхан.

O(1) санах ой — хэдэн ч элемент байсан цөөн хувьсагч

```
def niilber(items):
    total = 0                # ганц хувьсагч
    for x in items:
        total += x
    return total             # нэмэлт санах ой: O(1)
```

O(n) санах ой — оролттой адил хэмжээтэй шинэ бүтэц

```
def hoyor_dahin(items):
    result = []
    for x in items:
        result.append(x * 2)
    return result            # n элементтэй шинэ жагсаалт: O(n)
```

O(n) санах ой — рекурсийн стек

```python
def toololt(n):
    if n == 0:
        return 0
    return 1 + toololt(n - 1)   # n ширхэг дуудлага зэрэг «хүлээж» стек дээр байна
```

Рекурс шинэ жагсаалт үүсгээгүй ч дуудлага бүр стек дээр зай эзэлдэг. Python рекурсийн гүнийг анхдагчаар ≈1000-аар хязгаарладаг тул `toololt(5000)` нь `RecursionError` өгнө — санах ойн төвөгшлийн бодит жишээ!

Нуугдмал санах ой

Код	Санах ой	Яагаад?
`items[1:]`	O(k)	Зүсэлт шинэ хуулбар үүсгэнэ (k = зүссэн хэсгийн урт).
`[x * 2 for x in items]`	O(n)	List comprehension шинэ жагсаалт үүсгэнэ.
`(x * 2 for x in items)`	O(1)	Генератор: нэг удаад зөвхөн нэг утга.
`sorted(items)`	O(n)	Шинэ эрэмбэлэгдсэн жагсаалт буцаана.
`items.sort()`	≈ O(1)*	Байрандаа эрэмбэлнэ (шинэ жагсаалт үүсгэхгүй).

* Timsort дотроо түр санах ой (хамгийн муу үед n/2 хүртэл) ашиглаж болох ч шинэ жагсаалт буцаадаггүй.

07 Цаг ба санах ойн солилцоо

Ихэнхдээ **нэмэлт санах ой зарцуулж хугацаа хэмнэж** болдог. 3-р бүлгийн «давхардсан элемент байгаа юу?» асуудлыг өөр аргаар бодъё:

```python
def davhardsan_bii_yu_hurdan(items):
    harsan = set()              # аль хэдийн харсан утгууд
    for x in items:             # n удаа
        if x in harsan:         # set-ээс хайх: O(1)
            return True
        harsan.add(x)           # O(1)
    return False
```

Арга	Цаг	Санах ой	n = 100 000 үед
А: хоёр давталт (3-р бүлэг)	■ **O(n²)**	■ **O(1)**	≈ 5 000 000 000 харьцуулалт
Б: set ашиглах	■ **O(n)**	■ **O(n)**	≈ 100 000 шалгалт

Б арга 100 000 утга хадгалах санах ой «зардал» гаргаж, оронд нь 50 000 дахин бага алхам хийж байна. Энэ бол маш ашигтай наймаа.

Өөр нэг жишээ: «санаж авах» (memoization)

3-р бүлгийн `fib` нэг ижил утгыг дахин дахин боддог байсан. Бодсон хариугаа санаж авбал:

```python
from functools import lru_cache

@lru_cache(maxsize=None)          # бодсон хариуг санах ойд хадгална
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

# Цаг: O(2ⁿ) → O(n)       Санах ой: O(n)
```

Манай хэмжилтээр **fib(30)**: энгийн хувилбар **0.07 сек**, санах ойтой хувилбар **23 микросекунд**. fib(40) нь ≈123 дахин олон дуудлага хийдэг тул энгийн хувилбар ≈ **8 сек** болох бол санах ойтой нь мөн л агшин зуур.

> **Дүгнэлт**
> Санах ой хангалттай бол хурдыг сонгох нь ихэвчлэн зөв. Гэхдээ маш том өгөгдөл дээр санах ой дуусах эрсдэлтэйг үргэлж бодоорой.

08 Python үйлдлүүдийн хуудас

Өдөр тутам хэрэглэдэг үйлдлүүдийн цагийн төвөгшил. Нуугдмал O(n)-ийг олоход хэрэгтэй.

Үйлдэл	Код	Цаг	Тайлбар
list (жагсаалт)			
Индексээр авах	`a[i]`	**O(1)**	Байрлал нь шууд тооцоологдоно
Төгсгөлд нэмэх	`a.append(x)`	**O(1)***	
Төгсгөлөөс авах	`a.pop()`	**O(1)**	
Эхнээс авах	`a.pop(0)`	**O(n)**	Бусад бүх элемент нэг байр шилжинэ
Эхэнд оруулах	`a.insert(0, x)`	**O(n)**	Мөн адил шилжүүлэлт
Хайх	`x in a`	**O(n)**	Нэг бүрчлэн шалгана
Урт	`len(a)`	**O(1)**	Урт нь хадгалагдсан байдаг
Зүсэх	`a[i:j]`	**O(k)**	k = j − i; шинэ хуулбар
Эрэмбэлэх	`a.sort()`	**O(n log n)**	
Нийлбэр, хамгийн их/бага	`sum(a), max(a)`	**O(n)**	
set ба dict			
Хайх	`x in s, k in d`	**O(1)***	hash ашиглана
Нэмэх	`s.add(x), d[k] = v`	**O(1)***	
Түлхүүрээр авах	`d[k]`	**O(1)***	

Үйлдэл	Код	Цаг	Тайлбар
collections.deque			
Хоёр талаас нэмэх/авах	`dq.appendleft(x)`, `dq.popleft()`	■ **O(1)**	Дараалалд (queue) list-ийн оронд хэрэглэ

* Дундаж тохиолдолд. Ховор тохиолдолд (жишээ нь list дотоод санах ойгоо томруулах үед) нэг үйлдэл удаан байж болох ч нийтдээ O(1) гэж тооцно.

09 Түгээмэл андуурал

Андуурал 1: «O(1) гэдэг нь маш хурдан гэсэн үг»

Үгүй. O(1) гэдэг нь **n-ээс хамаарахгүй** гэсэн үг. Үргэлж 2 секунд зарцуулдаг үйлдэл ч O(1).

Андуурал 2: «O(n) үргэлж O(n²)-аас хурдан»

Том n дээр тийм. Харин жижиг n дээр тогтмол коэффициентээс шалтгаалан эсрэгээрээ байж болно. Python-ий sort хүртэл жижиг хэсгүүдийг O(n²) «оруулах эрэмбэлэлт»-ээр эрэмбэлдэг — учир нь жижиг n дээр тэр нь илүү хурдан.

Андуурал 3: «Ганц л давталт байгаа тул O(n)»

Давталт дотор **нуугдмал давталт** байж болно: list дээрх `in`, `.index()`, `.count()`, `insert(0, x)`, зүсэлт.

```
def niitleg(a, b):
    result = []
    for x in a:            # n удаа
        if x in b:         # b нь list бол → бас n удаа!   →  O(n²)
            result.append(x)
    return result

def niitleg_hurdan(a, b):
    b_set = set(b)         # O(n) — нэг удаа хөрвүүлнэ
    return [x for x in a if x in b_set]   # O(n) · O(1)  →  O(n)
```

Андуурал 4: «Big O = яг хугацаа»

Big O нь өсөлтийн **дээд хязгаар** бөгөөд ихэвчлэн **хамгийн муу тохиолдлыг** хэлдэг. Шугаман хайлт азтай бол 1 алхамд олно, гэхдээ хамгийн муу үед n алхам → O(n).

10 Дасгал

Функц бүрийн **цагийн** төвөгшлийг тодорхойл (n = оролтын урт). 5-р дасгалд **санах ойн** төвөгшлийг ч бод. Хариуг дараагийн хуудаснаас шалга — эхлээд өөрөө бодоорой!

Дасгал 1

```
def f1(items):
    return len(items) > 0
```

Дасгал 2

```
def f2(items):
    for x in items:
        print(x)
    for x in items:
        print(x * 2)
```

Дасгал 3

```
def f3(n):
    i = 1
    while i < n:
        i = i * 2
```

Дасгал 4

```
def f4(a, b):
    for x in a:
        for y in b:
            print(x, y)
```

Дасгал 5

```
def f5(items):
    result = []
    for x in items:
        if x not in result:
            result.append(x)
    return result
```

Дасгал 6

```
def f6(items):
    items.sort()
    return items[len(items) // 2]
```

Дасгалын хариу

#	Хариу	Тайлбар
1	■ **O(1)**	`len()` жагсаалтын уртыг хадгалж байдаг тул шууд буцаана.
2	■ **O(n)**	Дараалсан хоёр давталт: n + n = 2n → коэффициентийг хаявал O(n).
3	■ **O(log n)**	i нь 1, 2, 4, 8, … гэж **2 дахин** өсөж n-д хүрнэ → $\log_2$ n удаа.
4	■ **O(a · b)**	Гадна давталт a удаа, дотор нь b удаа. a = b = n бол O(n²).
5	■ **O(n²)**	`x not in result` нь list дотор **нуугдмал давталт**. Санах ой: O(n). Сайжруулалт: харсан утгаа set-д хадгалвал цаг O(n) болно.
6	■ **O(n log n)**	`sort()` давамгайлна; индексээр авах нь O(1). n log n + 1 → O(n log n).

11 Хураангуй

Сайнаас муу руу:

O(1)	O(log n)	O(n)	O(n log n)	O(n²)	O(2ⁿ)
Гайхалтай	Маш сайн	Сайн	Боломжийн	Том n дээр удаан	Зөвхөн жижиг n

Код харахдаа өөрөөсөө асуух 4 асуулт

1. **Давталт хэдэн удаа эргэх вэ?** n удаа → O(n); алхам бүрт хагасаар багасвал → O(log n).
2. **Давталт дотор давталт байна уу?** Байвал үржүүл → O(n²), O(a · b).
3. **Дуудаж буй функц дотроо давталттай юу?** `x in list`, `sort()`, зүсэлт, `insert(0, x)`.
4. **Шинэ бүтэц үүсгэж байна уу, рекурс хэр гүн очих вэ?** → санах ойн төвөгшил.

Санаж үлдэх зүйлс

- Big O бол **өсөлтийн хэлбэр**, секунд биш.
- Тогтмол коэффициент, жижиг гишүүнийг **хаяна**.
- Дараалсан хэсгийг **нэмнэ**, үүрлэсэн хэсгийг **үржүүлнэ**.
- «n 2 дахин томорвол юу болох вэ?» гэсэн **2 дахин тест** бол хамгийн хүчтэй зөн совин.
- Хайлт олон бол **set/dict**; эрэмбэлэгдсэн өгөгдөл бол **binary search**.
- Хурд ба санах ой хоёрын хооронд ихэвчлэн **солилцоо** бий.